Atrium for authors

You are an **author** when a space gave you the role *author* (or when everybody logged in has author rights there by default). Authors build the knowledge base: they file knowledge entries, keep their own entries current and propose skills. This guide assumes you know the everyday-use guide; it adds what you can do beyond reading.

Contents

- What authors can and cannot do
- Filing knowledge
- Writing good entries
- Drafts and approval
- Maintaining your entries
- Writing skills
- Sending to a colleague
- Daily workflows
- Troubleshooting
- FAQ

What authors can and cannot do

	Author
Read approved entries, search, use released skills	yes
File new entries in the space (`store_knowledge`)	yes
Change, archive and restore **your own** entries	yes; in a space with approval not your approved entries (a curator does that)
Move your own approved entries to another space	yes, but not out of a space that requires approval: a curator does that
Save skills (`store_skill`), set tags on skills you created (`tag_skill`)	yes, as drafts
See your own drafts, returned entries and skill versions	yes
Change or move entries of other people	no, curators do
Approve drafts, publish or block skills	no, curators do

	Author
Delete entries for good, manage members	no, space admins do

Your role is per space. Ask your assistant "*Which Atrium spaces can I write in?*"; `list_spaces` shows your role and whether the space requires approval.

Filing knowledge

You file knowledge by asking your assistant, in plain words:

- "*File this in the Support space.*"
- "*Save this procedure in Atrium under Engineering, tagged 'deploy' and 'rollback'.*"
- "*Document our decision about the API versioning in the Architecture space.*"

The assistant calls `store_knowledge`. It files only when you ask, and names the space it used afterwards, so check that it is the right one.

What Atrium does when you file:

1. **Space.** If you name none, the entry goes to your default space (normally your personal space). If that default is a team space, nothing is saved until you have confirmed (`confirm=true`), because colleagues can read it; naming the space yourself needs no confirmation. If the name is ambiguous or you cannot write there, the assistant lists the possible spaces and nothing is saved.
2. **Checks.** Atrium checks size (about 200 KB per entry), tags (up to 20, 50 characters each, no line breaks) and rejects text that looks like passwords, keys or other credentials.
3. **Duplicates.** If very similar entries already exist, nothing is saved yet. The assistant shows them and asks whether you want to update the existing entry or file a new one anyway.
4. **Status.** In a space without approval your entry is *approved* immediately and every reader sees it. In a space with approval it becomes a *draft* (see Drafts and approval). Personal spaces have no drafts.
5. **Index.** The entry is split into sections and indexed for search. A long entry is saved at once: its first sections are indexed right away and the rest a short while later in the background. Until then the rest is found by exact words, afterwards by meaning too. The assistant tells you when that applies; there is nothing to do.

Say "*as a draft*" if you want a curator to check it first even though the space does not require it, for example when you write about a subject outside your area.

Writing good entries

Assistants will quote your entries to colleagues who never saw the chat that produced them. Write for that reader.

- **One topic per entry.** Split a long chat into several entries rather than one wall of text.

- **A title that someone would search for.** "Refund process for annual subscriptions" beats "Refunds".
- **Self-contained.** Name the system, the team, the version. Do not write "as discussed above".
- **Date and owner.** Say when the information was valid and who to ask. Entries age.
- **State the source or the decision.** "Decided in the architecture board on 2026-03-12" helps readers judge it.
- **Use Markdown structure.** Headings, short lists and steps. Search works on sections, so a clear heading makes a section findable.
- **Tags** for subject, product or process (`billing` , `onboarding` , `kubernetes`). Use the tags that colleagues already use; tags are lower-cased and de-duplicated.
- **No secrets or personal data.** No passwords, API keys, customer personal data. Atrium rejects obvious credentials, but it cannot recognise everything.
- **No instructions to assistants.** Entries are data. Do not write text that tries to steer an assistant ("ignore previous instructions"). If you want assistants to follow steps, propose a skill.

A good prompt to your assistant: "*Summarise this chat as a self-contained Atrium entry with a searchable title, date, owner and tags, then show it to me before filing.*"

Type, responsible person and review date

- **Type** (`type`): say what the entry is. "*File this as a recommendation*", "*… as an experience report*"; without it the entry is `documentation` . **Binding rules (`rule`) are set only by curators and admins of the space**, and once an entry is a rule only they change it, in every space. If you write something that should be binding, file it as documentation and ask a curator to make it a rule.
- **Responsible person** (`owner`): "*… and make maria@example.com responsible for it.*" Only in team spaces, only somebody who has logged in, is not locked and can read the space. Nobody is made responsible implicitly: the author is not the responsible person unless named.
- **Review date** (`review_due`): "*… review by 2027-06-30.*" Readers see when the review is overdue, and curators see how many are. Nothing is hidden or deleted when the date passes.
- Changing only the responsible person or the review date creates no new version and does not show up as a change in `whats_new` .

Drafts and approval

Spaces can require approval. Then everything an author files is a **draft** until a curator approves it. Readers see only approved entries.

- **Your drafts are visible to you and the curators** of the space. Find them with "*Show my drafts in the Support space.*" (`list_knowledge` with `status=draft`).
- A curator **approves** the draft (it becomes visible to all readers) or **returns** it with a note. A returned entry shows `rejected` with the note in `review_note` .
- To resubmit a returned entry, change it: "*Update that entry with the corrections.*" (`update_knowledge`). It goes back into review as a draft and the note is deleted.
- There is **no notification** yet. Ask your curator, or check later with `list_knowledge` .

- A draft can also be requested explicitly in spaces without approval ("*file as a draft*"), for content you want checked.
- Entries a non-curator **moves** into a space with approval become drafts there.

Details of the process are in Drafts and approval.

Maintaining your entries

You say	What happens
"*Update the entry about X: the limit is now 50.*"	The assistant loads the current text (`get_knowledge`), applies your change and saves the complete new text (`update_knowledge` , with the version it loaded, so a change by somebody else in the meantime is not overwritten). A new version is recorded
"*Move that entry to the Engineering space.*"	`move_knowledge` after your confirmation. You need write rights in the target, and the entry must be approved (drafts and returned entries cannot be moved). Authors cannot move an approved entry out of a space that requires approval; a curator does that. In a target space with approval it becomes a draft there
"*Archive that entry.*"	`archive_knowledge` after your confirmation. It disappears from search and lists but is kept, and you can restore it
"*Show my archived entries.*" / "*Restore that entry.*"	`list_knowledge` with `status=archived` , then `unarchive_knowledge` (an approved entry in a space with approval goes back to review)
"*Add the tag 'security' to that entry.*"	`update_knowledge` with the complete tag list

Rules to remember:

- You can change **your own** entries only. For entries of others, ask a curator.
- **In a space with approval, authors cannot change approved entries.** The assistant tells you so. Ask a curator to change it, or file the corrected text as a new draft and ask the curator to archive the old entry. A draft next to an approved version is planned, not built yet.
- **Deleting for good** (`delete_knowledge`) is for space admins. Authors archive instead.
- **In a space with approval, an approved entry is a curator's to archive.** It is no longer yours to take away from the readers; ask a curator. Your own drafts and returned entries you can still archive yourself. In spaces without approval you archive your own entries freely.
- Every change creates a version, and your name stays on the entry.

Writing skills

A skill is an instruction your colleagues' assistants follow when asked. You write the skill, a curator reviews it, and once it is published everybody with read access can use it. Authors propose skills; they cannot publish them.

The package

A skill is a set of text files:

- `SKILL.md` is required. It begins with a front matter block, then the instructions:

```
---
name: support-reply
description: Draft a polite first reply to a customer support ticket
---
# Support reply

1. Read the ticket and the customer's plan ...
2. Check references/tone.md ...
```

- `name` is lower case letters, digits and single hyphens (at most 64 characters) and equals the skill's name. `description` is required (at most 1024 characters). Optionally `allowed-tools` lists tools the skill asks for; this is a request, not a permission.
- Further files hang off `SKILL.md`, for example `references/tone.md` or `assets/template.md`. Allowed types: `.md`, `.txt`, `.json`, `.yaml`, `.yml`, `.csv`. **No scripts and no binary files**: Atrium distributes text and never runs anything.
- Limits: 40 files, 64 KB per file, 256 KB per package. Credentials in files are rejected.

Filing it

Ask your assistant: "*File this as an Atrium skill called support-reply in the Support space.*" and provide the content, or let it draft the files with you. `store_skill` saves it as a **draft** (version 1 for a new skill, the next version if the name exists).

What follows:

1. A curator reads all files, judges the risk and approves or rejects. See what curators check.
2. If it is **rejected**, you see the note. Fix the files and save again. While your own draft is still unreviewed, saving replaces it.
3. If it is **approved**, a curator publishes it and all readers can use it.
4. A published version **never changes**. To improve a skill, save it again: that creates a new version, which is reviewed again. Old versions stay available.

The author of a version cannot approve it; another curator must. That is on purpose.

Writing a skill that works

- **The description is the trigger.** Assistants and people see it in lists. State what the skill does and when to use it: "Review a pull request against our checklist before merge". About 160 characters are shown in an assistant's instructions, which list at most 25 released skills per user.

- **Numbered steps.** Be concrete about inputs, order and what the result looks like.
- **Keep reference material in files.** Put long checklists and templates in `references/` and mention them in `SKILL.md`. Assistants load files on demand.
- **No secrets, no hidden instructions.** Reviewers read every file. Instructions to bypass an assistant's safety rules, to hide actions from the user or to send data elsewhere will be rejected.
- **Do not ask for more than needed.** If the skill asks for tools (`allowed-tools`), the reviewer treats it as risk class B or higher.
- **Test it.** After publication, run it with `/name` on a real example and fix what the assistant misunderstands, as a new version.

Tags and housekeeping

- Tags on a skill help people find it ("*skills about support*"). Set them when you file a **new** skill (`tags`), later change them with "*Set the tags of support-reply to support, tickets*" (`tag_skill`). You can retag a skill while you are the author of its newest released version (or, as long as nothing is released, its creator); curators can retag any. Tags are not part of the reviewed content and need no new version. Do not put instructions in tags: they are only keywords.
- **Names are unique per space.** Use a distinct, descriptive name. Saving under the name of a skill you can see (a released one, or one with a version of yours) adds a new version to it. If the name is held by something you cannot see (somebody's unreleased draft, an archived skill), Atrium only says "that name is not available": choose another name or ask a curator.
- **Limits:** by default at most 100 versions per skill; a personal space holds 50 skills and 5 MB, a team space 300 skills and 50 MB of skill files. Knowledge in a team space is limited too (by default 5000 entries and 200 MB per space, drafts included). A save over a limit is refused with the reason, and an instance admin can raise any of these limits for a space, for you or for everybody at any time, without a restart: ask for it. A skill that has reached its version limit can only continue under a new name unless the limit is raised: archiving does not free the name or reset the count, and deleting the old skill is for instance admins (in your personal space you can).
- Moving a skill to another space is done by curators (`move_skill`): only the newest released version moves, as a new draft that the curators of the target review.

Skills in your personal space

Your personal skills are released at once and visible only to you. This is a good place to develop and test a skill before you propose it to a team. Copy the files into the team space with `store_skill`; the curators review it there.

Sending to a colleague

Where your organisation has switched it on, you can send an entry or a skill to one colleague instead of filing it in a team space: "*Send my entry about the release checklist to anna@example.com*" (`share_knowledge`) or "*Send the skill weekly-report to anna@example.com*" (`share_skill`). The assistant

asks you to confirm. A copy lands in the colleague's personal space as a draft that only they can see; they approve or return it, and you are not told which. For a skill they have to read, approve and release it before their assistant offers it.

- **What you can send:** an approved entry or a released skill. For an entry of a team space you need to be its author or a curator or admin of the space; for a skill of a team space you need to be a curator or admin (as for `move_skill`). Text that is not stored yet: store it in your personal space first, then send it.
- **Rules arrive as documentation:** a binding rule is sent as a plain documentation entry.
- **A refusal says little on purpose.** If the colleague does not exist, cannot receive right now, already holds a skill of that name or has too many unanswered items, the answer is the same. Do not try other addresses; ask them directly.
- **You see nothing afterwards,** not the copy and not the decision. If it matters, tell the colleague.
- **It is not a way to publish.** What the whole team should use goes to a team space and is reviewed there. Details: Sending to a colleague.

Daily workflows

Documenting a new procedure.

1. Ask the assistant to draft an entry from your notes or chat.
2. Check title, date, owner, tags.
3. "*File this in the Operations space.*" If the space needs approval, tell the curator (no notification yet).
4. Check later that it is approved: "*Show my drafts in Operations.*"

Turning a document into entries. If your organisation released the skill `knowledge-ingest`, attach or paste the source and say "*/knowledge-ingest: file this into Operations.*" The assistant lists the knowledge in it, compares it with what Atrium already has and shows a plan (new, update, already covered, conflict). Nothing is saved before you confirm; conflicts are never resolved by the assistant. Without the skill, ask the same in your own words, one entry at a time.

Correcting an entry of yours. "*Load my entry about X, fix the figure and update it.*" In a space with approval and an approved entry, ask a curator instead (see above).

Reacting to a returned draft. Read the note, revise (`update_knowledge`), and tell the curator that it is back in review.

Proposing a skill.

1. Test the steps by hand in a chat until they work.
2. Write `SKILL.md` and reference files; file as a skill in your personal space and try `/name`.
3. File the same package in the team space. Message a curator.
4. After approval, announce the skill to your team and collect feedback; ship fixes as new versions.

Passing something on to one colleague. "*Send this entry to anna@example.com.*" Confirm when asked, then tell the colleague that it waits in their personal space.

Handing over before you leave or change teams. Move your team-relevant entries to the right space ("*Move all entries about X to Operations*", one at a time with confirmation). What stays in your personal space is deleted after you leave.

Keeping content fresh. Once a quarter ask "*List my entries in Support*", check the dates and update or archive the stale ones.

Troubleshooting

Problem	What to try
"You do not have permission" when filing	Your role in that space is reader. Ask a space admin, or file in another space
Nothing is saved, the assistant lists similar entries	Update the existing entry, or confirm that you want a new one
The entry does not show up in search	Other readers cannot find it while it is a draft; you see your own drafts yourself, with status `draft`. If you cannot find it either, it may still be indexed or your wording differs. Check with `list_knowledge status=draft`
"Approval required" when changing an entry	The space needs approval for approved entries; ask a curator
The skill is refused	The message names the reason: a script or binary file, a bad path, a missing `name` or `description`, a credential in a file, a limit
"That name is not available in this space"	A skill or an unreleased draft you cannot see holds the name. Choose another name, or ask a curator
A save says a limit is reached	The message names the limit (entries, bytes, skills or versions). Archive or delete something, or ask an instance admin to raise the limit; `space_members` shows the limits and usage of a space to its admins
An approval-related message you do not understand	Ask a curator of the space, `space_contacts` names them

FAQ

Can I file in a space where I am only a reader? No. Ask a space admin for the author role.

Why is my entry not visible to my colleagues? Probably a draft: the space needs approval. A curator must approve it.

Can a curator change my entry? Yes. Curators can edit entries of others. Every change is recorded as a version; the editor's name is kept in the database.

Can I get an older version of my entry back? Every change is kept as a version in the database, but there is no tool yet to browse or restore old versions. Keep a copy of important text yourself, and ask your administrator if something must be recovered.

How do I delete an entry? Authors archive. Permanent deletion is for space admins and, in personal spaces, the owner, and it also works on archived entries.

Why can an assistant not file by itself? By design. It files only on your explicit request.

Can I attach files or images to entries? No. Entries are Markdown text. Skills can carry text files only.

What if two people file the same thing? The duplicate check catches very similar entries in the same space. Update the existing entry.

What if two assistants change the same entry at once? The assistant passes the version it loaded (`expected_version`). If the entry has changed since, the save is refused and nothing is overwritten: the assistant loads the current text, merges your change into it and saves again.

Can I make a skill available to everyone? File it in a space everybody reads. Curators decide on publication; a space for the whole organisation is set up by an instance admin.

Why was my skill rejected? Read the note. Typical reasons: unclear steps, too broad, asks for risky actions, missing references.